Clase 34 · Árboles de decisión

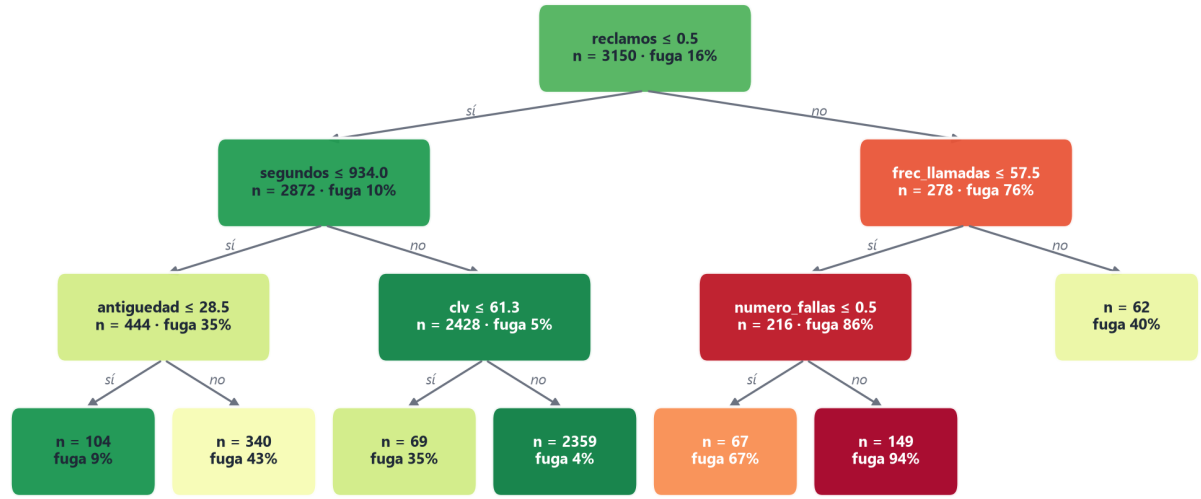
Profesor Eduardo Rodríguez · Etapa 7 del roadmap · misma base de fuga de la Clase 33

La gran idea de la clase

Un árbol parte el espacio con preguntas de sí o no y, en cada hoja, predice la tasa de su grupo. Es «la técnica más simple y a la vez más peligrosa», en palabras del profesor: la más fácil de explicar y la más fácil de **sobreajustar**. Se elige cuando la explicación vale más que unos puntos de AUC.

El árbol de la clase

Árbol de profundidad 3 (min_samples_leaf = 60) con los datos de la clase. Color: rojo = más fuga.



Lo que necesitas ANTES

- Limpieza y ganancia de información (Clase 33)
- Validación cruzada y AUC (Clases 30–33)
- Regresión logística, para comparar (Carpeta 2)
- Proporciones y logaritmo en base 2

Tres cosas de la Clase 33 que hoy se explican solas

- La **ganancia de información** que usamos como filtro es el criterio con que el árbol elige cada corte
- La **importancia de un árbol** es selección embebida: el árbol elige variables al entrenar
- **reclamos** domina: es la raíz del árbol, con una ganancia que más que duplica a la segunda

Clase 34 · Cómo parte el espacio un árbol

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

La regresión logística ajusta un solo plano para todo el espacio. El árbol no ajusta ninguna fórmula: corta el espacio en rectángulos y en cada uno predice la proporción de fuga que observó.

Conceptos clave

1. **Regiones.** Cada hoja es una región R_m y su predicción es la tasa de fuga de los clientes que cayeron ahí.
2. **No linealidad e interacciones solas.** Una rama puede usar variables distintas a otra. Por eso la clase NO construye las 50 variables de la Clase 33: el árbol llega a lo mismo partiendo el espacio, y con 12 variables se puede leer.
3. **Segmentos, no coeficientes.** La hoja más roja tiene 149 clientes con 94% de fuga: «reclamó, usa poco el servicio y tuvo fallas». Es un segmento al que se le puede mandar una campaña mañana.
4. **Campañas de precisión.** La hoja de 69 clientes (2.2% de la base) con 35% de fuga sirve para una campaña de bajo presupuesto. Cruzarla con el CLV dice si vale la pena retener a cada cliente.
5. **Sin escalar.** El árbol solo pregunta «¿x ≤ umbral?». No necesita estandarizar ni transformar las variables.

Fórmulas y reglas

Logística: $\log(p / (1 - p)) = \beta_0 + \beta_1 x_1 + \dots$

Árbol: $\hat{p}(x) = \Sigma_m c_m \cdot 1\{x \in R_m\}$

c_m = tasa de fuga en la hoja m

lift = tasa de la hoja / tasa general

En Python

```
arbol = DecisionTreeClassifier(
    max_depth=3, min_samples_leaf=60,
    random_state=0).fit(X, y)
export_text(arbol,
feature_names=list(X.columns))
hoja = arbol.apply(X) # hoja de cada cliente
```

Con los datos de la clase

- 7 hojas, profundidad 3, mínimo 60 clientes por hoja
- Segmentos: 149 → 94% · 67 → 67% · 340 → 43% · 62 → 40% · 69 → 35% · 104 → 9% · 2.359 → 4.5%
- La hoja de 94% tiene un lift de 6: seis veces la tasa general (15.7%)

Cuidado
min_samples_leaf = 60 no es un detalle: con hojas de 5 clientes, una tasa de 100% puede ser puro ruido de muestra chica.

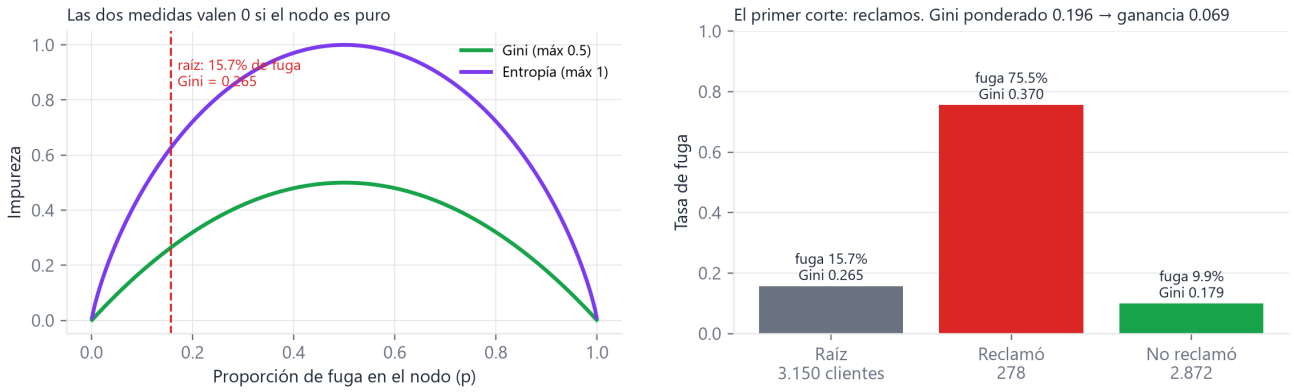
Clase 34 · El criterio de corte: Gini y entropía

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

En cada nodo, el árbol prueba todas las variables y todos los umbrales, y se queda con el corte que más reduce la mezcla de clases. Es un algoritmo voraz: elige lo mejor ahora, sin mirar dos pasos adelante.

Conceptos clave

1. **Impureza.** Mide qué tan mezclado está un nodo. Vale 0 si el nodo es puro y es máxima con 50/50 (Gini 0.5, entropía 1).
2. **Los p_i son del nodo.** Se recalculan en cada nodo con los clientes que caen ahí. En la raíz: 495 de 3.150 se fugan → Gini 0.265.
3. **Ganancia.** Impureza del padre menos la impureza ponderada de los hijos. Es la misma fórmula que usamos como filtro en la Clase 33.
4. **El barrido.** Para la raíz se evaluaron 378 combinaciones de variable y umbral. Y el barrido completo se repite en CADA nodo.
5. **Gini o entropía.** Suelen dar árboles parecidos; la entropía viene de la teoría de la información (la «medida del desorden»). Aquí, con profundidad 5: Gini 0.906 vs entropía 0.920.



Izquierda: las dos medidas según la proporción de fuga del nodo. Derecha: el primer corte real, reclamos, separa un grupo de 76% de fuga de otro de 10%.

Fórmulas y reglas

$$\text{Gini} = 1 - \Sigma\, p_i^2$$

$$\text{Entropía} = -\Sigma\, p_i \cdot \log_2 p_i$$

$$\text{ganancia} = \text{imp(padre)} - \Sigma\, (n_j/n) \cdot \text{imp(hijo}_j\text{)}$$

raíz: $1 - 0.843^2 - 0.157^2 = 0.265$

En Python

```
def gini(p): return 1 - p**2 - (1 - p)**2
DecisionTreeClassifier(criterion='entropy')
cross_val_score(arbol, X, y, cv=skf,
    scoring='roc_auc')
```

Con los datos de la clase

- Reclamó: 278 clientes, 210 fugan (75.5%), Gini 0.370
- No reclamó: 2.872, 285 fugan (9.9%), Gini 0.179
- Ponderado 0.196 → ganancia 0.069, más del doble que la segunda (estado, 0.027)
- Edad apenas gana 0.0005: sus hijos se parecen al padre

Cuidado
Voraz significa que no garantiza el árbol óptimo, solo uno bueno y rápido. Un corte malo arriba condiciona todo lo de abajo.

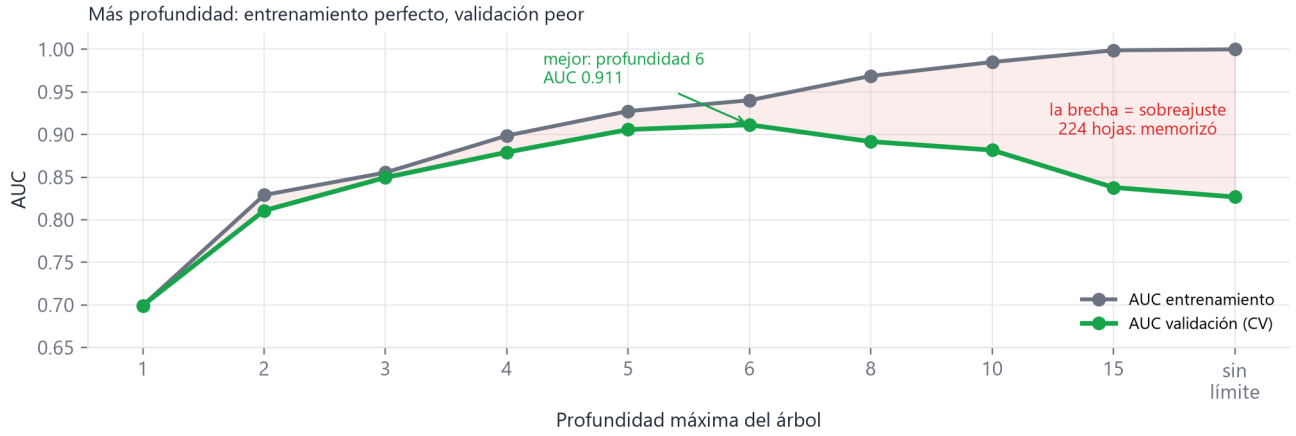
Clase 34 · Sobreajuste y validación cruzada

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

Un árbol sin límite crece hasta que cada hoja es pura: acierta todo el entrenamiento y memoriza. La curva de profundidad muestra exactamente dónde deja de aprender y empieza a memorizar.

Conceptos clave

1. **Train vs validación.** El AUC de entrenamiento siempre sube con la profundidad. El de validación sube hasta un punto y después baja: esa brecha es el sobreajuste.
2. **El punto de quiebre.** Con los datos de la clase, la mejor profundidad es 6 (AUC 0.911). Sin límite, el árbol tiene 224 hojas, AUC de entrenamiento 1.000 y de validación 0.827.
3. **Validación cruzada.** 5 bloques de 630 clientes: se entrena con 2.520 y se mide en el bloque restante, rotando. Importa el promedio Y el desvío entre rondas.
4. **Separar al azar, no por fecha.** Partir train y test por fecha sin querer es «un error gravísimo, pero pasa». Estratifica para mantener el 15.7% de fuga en cada bloque.
5. **Tolerancia práctica.** El profesor acepta como mucho unos 2 puntos de diferencia entre entrenamiento y validación. En la industria, un AUC de 0.81–0.85 en fuga ya es bueno.



Curva real con los datos de la clase. El área rosada es la brecha entre lo que el árbol memoriza y lo que generaliza.

Fórmulas y reglas

sobreajuste = AUC train − AUC validación

k-fold: k modelos, cada uno validado en 1/k de los datos

elegir la profundidad con el MÁXIMO AUC de validación

En Python

```
skf = StratifiedKFold(5, shuffle=True,
random_state=0)
cross_validate(arbol, X, y, cv=skf,
    scoring='roc_auc', return_train_score=True)
```

Con los datos de la clase

- Rondas (profundidad 5): 0.901 · 0.920 · 0.911 · 0.918 · 0.880
- Promedio 0.906 · desvío 0.015
- Profundidad 3 → 0.850 · 6 → 0.911 · 15 → 0.838

Cuidado
Un AUC de 0.90+ «demasiado bueno» en datos reales suele ser leakage, no un gran modelo. Si una sola variable explica casi todo, el profesor sugiere evaluar submodelos por segmento: la población puede no ser homogénea.

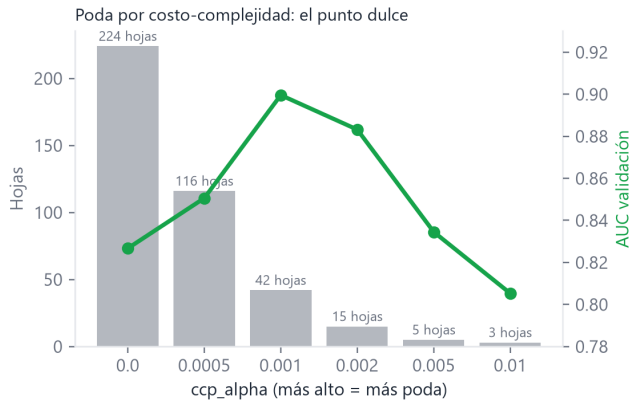
Clase 34 · Poda e importancia de variables

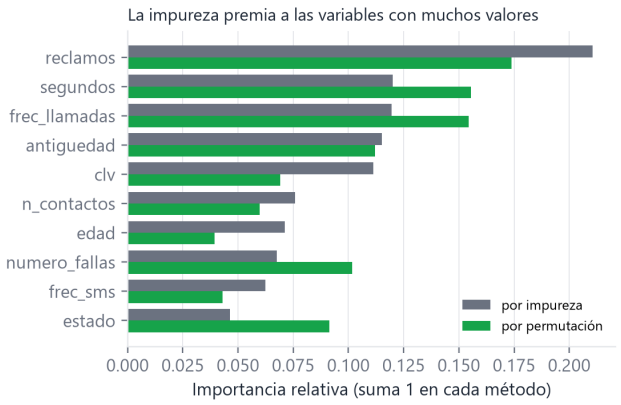
Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

Hay dos formas de frenar al árbol: antes (pre-poda, con límites) o después (post-poda, recortando ramas que no compensan). Y para saber qué variables importan, la importancia por impureza engaña: hay que usar permutación.

Conceptos clave

1. **Pre-poda.** Limita el crecimiento durante el entrenamiento: max_depth, min_samples_leaf, min_samples_split, max_leaf_nodes.
2. **Post-poda por costo-complejidad.** Deja crecer el árbol y lo recorta: cada hoja debe «pagar» α. Con α = 0 sobrevive todo; al subir α quedan menos hojas. α se elige por validación cruzada.
3. **El punto dulce.** Con los datos de la clase, α = 0.001 deja 42 hojas y AUC 0.900; la búsqueda del notebook elige α = 0.0013, con 36 hojas y AUC 0.909.
4. **Importancia por impureza (sesgada).** Premia a las variables con muchos valores distintos: tienen más umbrales para probar y, por azar, alguno reduce la impureza. clv tiene 2.654 valores.
5. **Importancia por permutación.** Desordena una variable en datos no vistos y mide cuánto cae el AUC. Ahí clv baja y estado (binaria) sube: la impureza premiaba la cardinalidad.





Izquierda: hojas y AUC según α (datos de la clase). Derecha: importancia relativa de un Random Forest por los dos métodos.

Fórmulas y reglas

$\text{costo}_\alpha(T) = R(T) + \alpha \cdot |\text{hojas}(T)|$

α = 0 → árbol completo · α ↑ → menos hojas

imp. permutación = AUC − AUC(con X_j desordenada)

En Python

```
path = arbol.cost_complexity_pruning_path(X, y)
GridSearchCV(DecisionTreeClassifier(),
    {'ccp_alpha': path.ccp_alphas[:-1]},
    cv=skf, scoring='roc_auc')
permutation_importance(modelo, X_test, y_test,
    n_repeats=20, scoring='roc_auc')
```

Con los datos de la clase

- α: 0 → 224 hojas, 0.827 · 0.001 → 42, 0.900 · 0.01 → 3, 0.805
- Impureza: reclamos 0.206 · segundos 0.118 · clv 0.109 · estado 0.045
- Permutación: reclamos 0.035 · segundos 0.032 · clv 0.014 · estado 0.019

Cuidado
La importancia por permutación se mide SIEMPRE sobre datos que el modelo no vio. Sobre el entrenamiento hereda el sobreajuste.

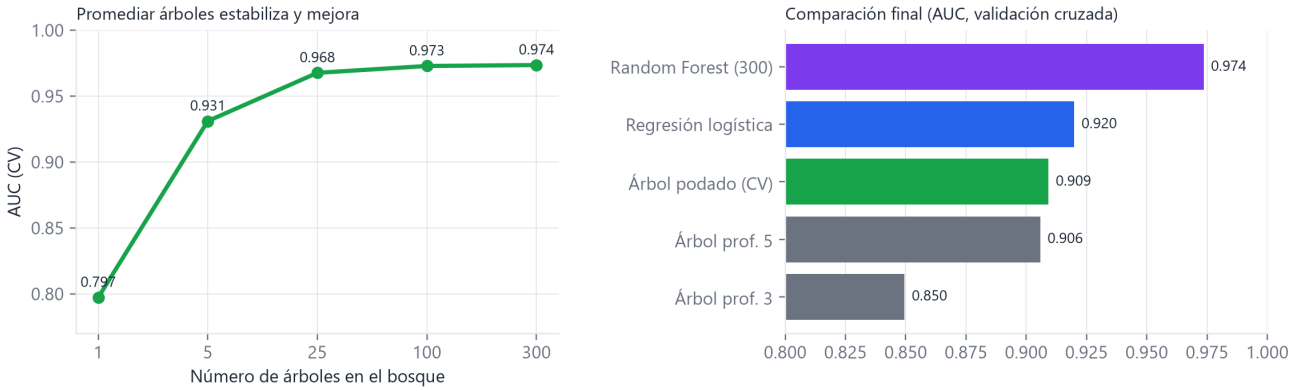
Clase 34 · Fortalezas, límites y el salto al bosque

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

El árbol se dibuja y se explica a cualquiera, pero es inestable y un árbol solo suele predecir peor que una logística. La solución a la inestabilidad es promediar muchos árboles: el Random Forest.

Conceptos clave

1. **Invariante a transformaciones monótonas.** Aplicar log o raíz a una variable mueve el umbral pero no cambia quién queda a cada lado: el AUC del árbol queda idéntico (0.906). La logística sí cambia (0.920 → 0.923).
2. **Inestable.** Con 6 muestras distintas, la raíz siempre es reclamos, pero el segundo nivel cambia (clv, segundos o frec_llamadas). Con reentrenamientos trimestrales, eso es explicar un árbol distinto cada vez.
3. **Otros límites.** Solo corta perpendicular a los ejes (una diagonal le cuesta muchos escalones) y predice constantes por región: no extrapola.
4. **El bosque.** Muchos árboles, cada uno con una muestra bootstrap y un subconjunto de variables por corte (√12 ≈ 3), promediados. Con 300 árboles: AUC 0.974.
5. **¿Para qué es el modelo?.** El árbol de profundidad 3 cabe en una lámina; el bosque no. Es el mismo trade-off de la Clase 32: se cede interpretabilidad a cambio de desempeño.



Izquierda: el AUC del bosque según el número de árboles. Derecha: todos los modelos de la clase, con validación cruzada.

Fórmulas y reglas

$\text{Var}(\text{promedio}) = \rho\sigma^2 + (1 - \rho)\sigma^2 / B$

B = n° de árboles · ρ = correlación entre árboles

max_features = √p decorrelaciona los árboles

En Python

```
RandomForestClassifier(n_estimators=300,
    max_features='sqrt', n_jobs=-1,
    random_state=0)
np.log1p(X.segundos) # no cambia el árbol
```

Con los datos de la clase

- Árboles en el bosque: 1 → 0.797 · 5 → 0.931 · 25 → 0.968 · 300 → 0.974
- Solo bootstrap 0.969 · bootstrap + subconjunto 0.974
- Logística 0.920 · árbol podado 0.909 · árbol prof. 3 0.850

Cuidado
El profesor confiesa su preferencia entre los modelos clásicos: Gradient Boosting (Clase 36). Hoy los ensambles son tan baratos de entrenar que un árbol solo se elige casi solo por explicabilidad.

Chuleta, hiperparámetros y checklist

Lo que vas a ajustar en cualquier modelo de árboles

Hiperparámetro	Qué controla	Rango típico	Efecto si lo subes
max_depth	preguntas encadenadas	3–8	más complejo, más sobreajuste
min_samples_leaf	clientes mínimos por hoja	5–60 (clase: 60)	más simple, tasas más estables
min_samples_split	clientes mínimos para partir	10–50	más simple
max_leaf_nodes	hojas máximas	10–50	más complejo
ccp_alpha	precio de cada hoja (post-poda)	0–0.05	más poda, menos hojas
criterion	medida de impureza	gini / entropy	—
class_weight	peso de la clase minoritaria	None / 'balanced'	más sensibilidad a la fuga

Rangos típicos tomados de la presentación genérica de la clase («Arbol de decision.pptx»).

Checklist para entrenar un árbol

	Paso	Cómo sé que lo hice bien
☐	Separa train y test al azar (y estratificado)	nunca por fecha sin querer
☐	Fija min_samples_leaf con criterio	hojas muy chicas dan tasas de ruido
☐	Traza la curva de profundidad	elige donde el AUC de validación es máximo
☐	Poda con ccp_alpha validado	GridSearchCV, no a ojo
☐	Revisa el desvío entre folds	un buen promedio con mucho desvío no es estable
☐	Mide importancia por permutación	sobre el test, nunca sobre el entrenamiento
☐	Traduce las hojas a reglas	n, tasa y lift de cada segmento
☐	Compara con una logística y un bosque	y decide según para qué es el modelo

Alertas del material, repaso y criterio de dominio

Diferencias entre materiales, preguntas clave y cuándo pasar a la Clase 35

Alertas del material

Errores y diferencias en las diapositivas (verificados)

- **Diapositiva 13, nota de respuesta:** dice que el candidato A (reclamos) da «Gini ponderado ~0,168 y ganancia ~0,097». Con los datos de la misma diapositiva da **0.195 y 0.070**, lo mismo que la diapositiva 12. Además, esa diapositiva usa 211 y 284 fugas, y los datos reales son 210 y 285.
- **Diapositiva 19:** muestra como resultado {'ccp_alpha': 0.001}, AUC 0.8996 y 42 hojas. El notebook, ejecutado con los datos, elige **α = 0.0013, AUC 0.909 y 36 hojas**: usa otra grilla de valores. El 0.8996 corresponde a la fila α = 0.001 de la tabla de la diapositiva 18.
- **Diapositivas 25 y 27:** una dice «el mejor árbol solo 0,911» y la otra «solo llega a 0,906». No se contradicen: 0.911 es la profundidad 6 y 0.906 la profundidad 5.
- **Diapositiva 11:** la fila «84% / 16% ← nuestra raíz» muestra Gini 0.269 porque redondea. El valor exacto es 0.265.

Otras observaciones

- El comentario del notebook dice que Gini y entropía «dan árboles casi idénticos», pero con profundidad 5 la entropía da 1.4 puntos más de AUC (0.920 vs 0.906). Aquí sí hay diferencia.
- El notebook carga `churn_telco.csv`, que no existe. Tu notebook guiado usa el Excel de la Clase 33.
- **La transcripción de esta clase tiene ruido:** bucles de Whisper (líneas 855–857 y 893–901), un fragmento ajeno a la clase sobre cómo pausar OBS Studio (líneas 1085–1101) y términos mal reconocidos: «Spirman» (Spearman), «la nova» (ANOVA), «vernugis» (Bernoulli), «araso» (Lasso), «Scorchart» (scorecard) y «Attention y Solve Unit» (el paper «Attention Is All You Need», 2017).

Preguntas de repaso

#	Pregunta	Respuesta corta
1	¿Por qué se dice que el árbol es «voraz»?	Elige el mejor corte en cada nodo sin mirar los siguientes: no garantiza el árbol óptimo.
2	Gini de un nodo con 15 de clase A y 5 de B	p = 0.75 → 1 − 0.5625 − 0.0625 = 0.375 (entropía 0.811).
3	¿Qué diferencia la pre-poda de la post-poda?	Pre-poda limita al crecer (max_depth, min_samples_leaf); post-poda recorta después con ccp_alpha.
4	¿Por qué engaña la importancia por impureza?	Premia variables con muchos valores; la permutación sobre el test lo corrige.
5	99% en train y 71% en test, ¿qué haces?	Regularizar (profundidad, hojas mínimas), podar con α validado y solo después probar un bosque.
6	¿Por qué el árbol no necesita escalar?	Solo pregunta x ≤ umbral: una transformación monótona mueve el umbral, no la partición.

Criterio de dominio para pasar a la Clase 35

- ☐ Calculo a mano Gini, entropía y la ganancia de un corte
- ☐ Leo un árbol y traduzco sus hojas a reglas con n, tasa y lift
- ☐ Diagnostico sobreajuste con la curva de profundidad
- ☐ Podo con ccp_alpha elegido por validación cruzada
- ☐ Explico por qué uso importancia por permutación
- ☐ Resuelvo los bloques 1 a 8 del cuaderno con al menos 80% de aciertos